

Humanoid robot control – practical aspects



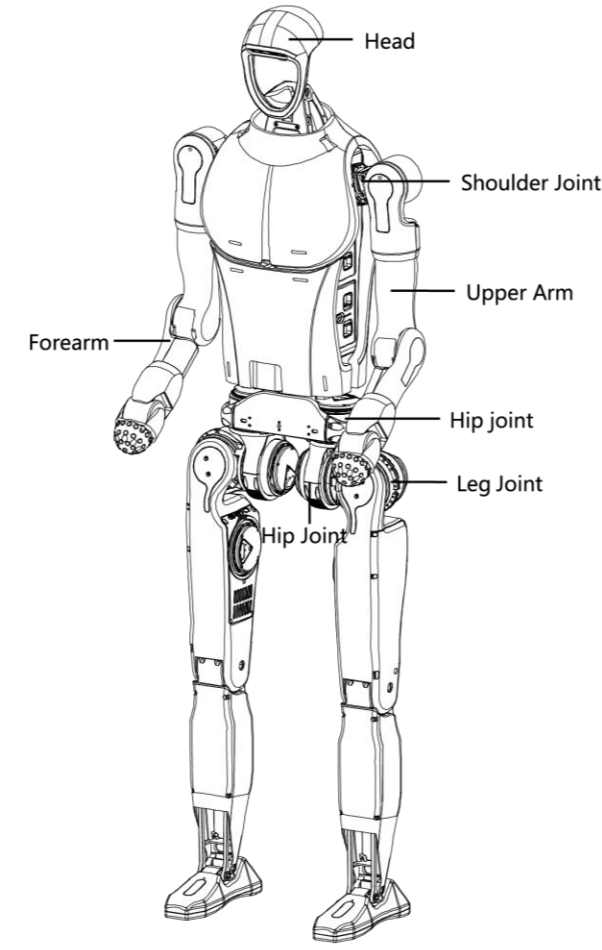
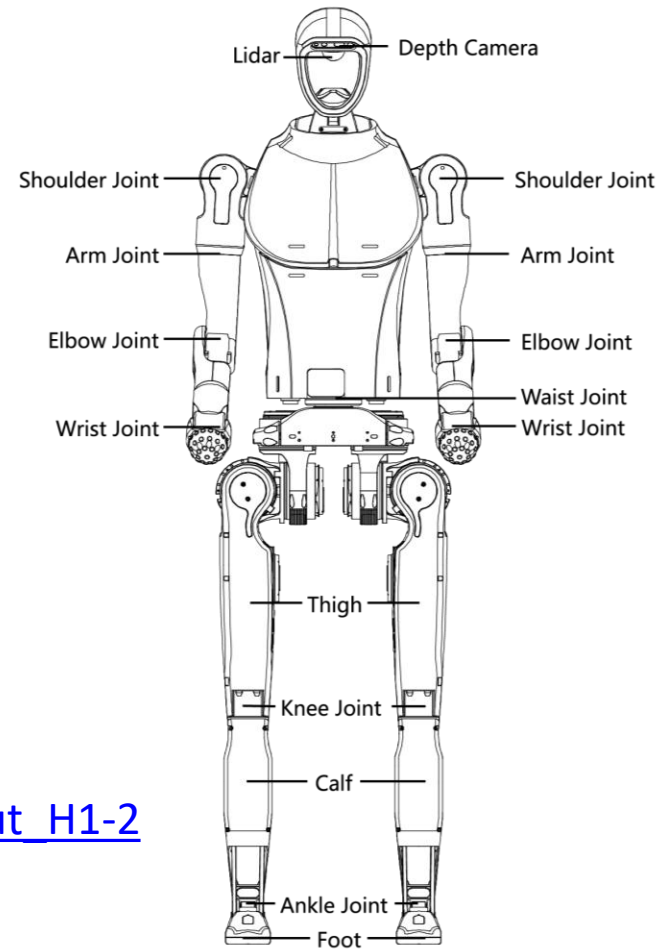
Field and Service Robotics

- Unitree H1-2
- Unitree_sdk2
- MuJoCo simulator
- Control interfaces
- Code example: kinematic planner



- Commercial humanoid robot
- Weight: 70 kg, Height: 1.70 m
- 27 Degrees of freedom
- Actuators: M107 Joint Motor
- Sensors: Imu, Lidar, RGB-D camera

https://support.unitree.com/home/en/H1_developer/About_H1-2



■ Remote control

- Human-controlled walking... and not much more!



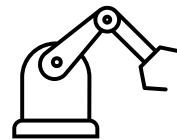
■ “High-level” API

- Autonomous walking and/or upper body control (arms/hands)



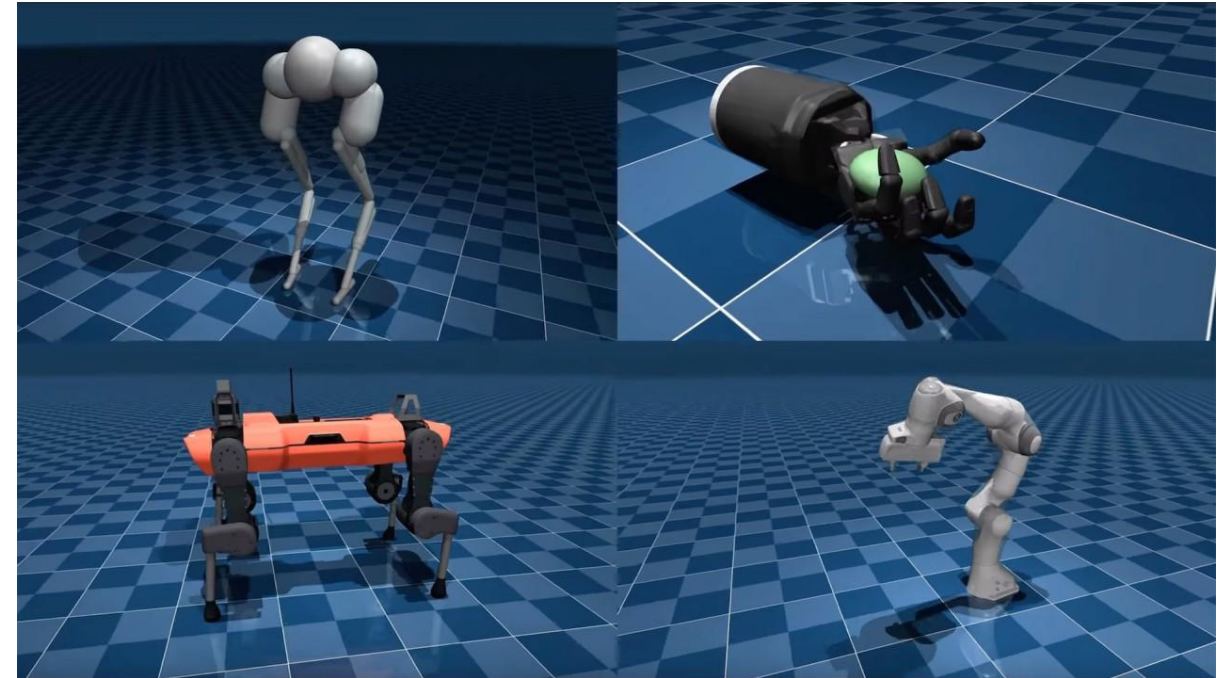
■ “Low-level” API

- Whole body control (PD control of all joints)

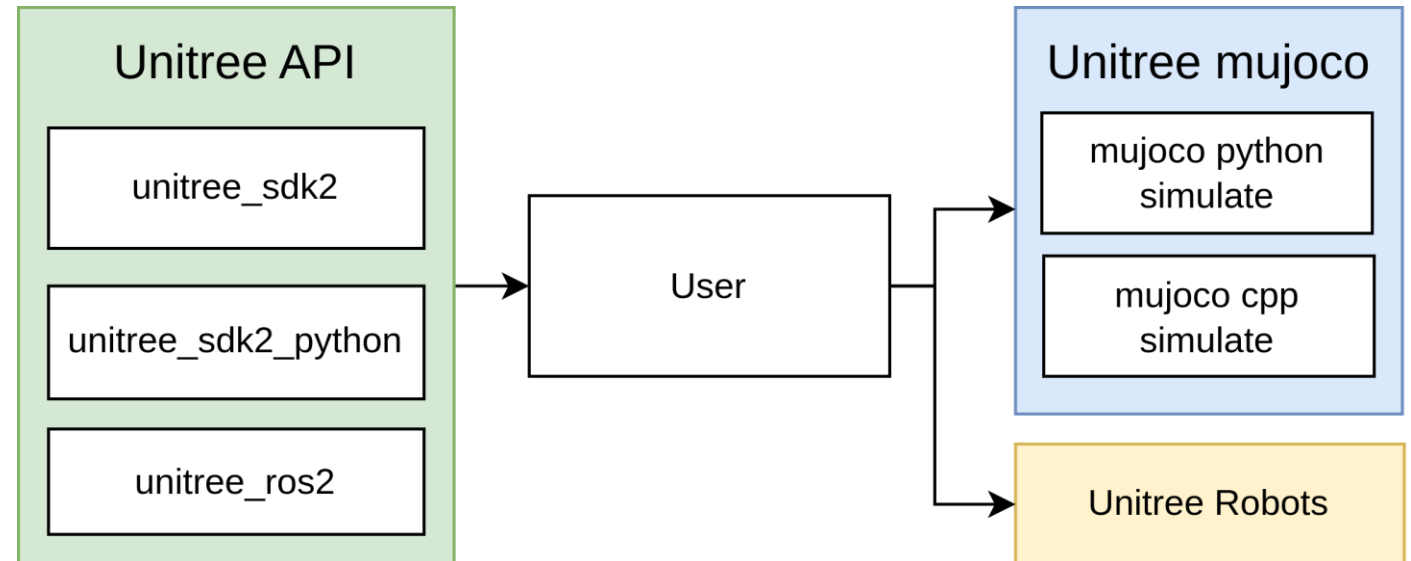


- https://github.com/unitreerobotics/unitree_sdk2
- This repository contains the open-source API used to control Unitree robots
- Control of Unitree robots via this sdk is based on a DDS system (topics with publisher and subscribers, similar to ROS2)
 - Example of supported messages are LowCmd and LowState for motor control commands and information on robot state
 - There is also the possibility of using ROS2 wrappers directly https://github.com/unitreerobotics/unitree_ros2
- Written in C++, but also a python-based repo is available:
https://github.com/unitreerobotics/unitree_sdk2_python
- We want the same code to run on simulation and on the real robot

- MuJoCo (**M**ulti-**J**oint dynamics with **C**ontact) is an open-source physics engine optimized for robot control
- Some differences with other simulators (such as Gazebo, Isaac, etc.):
 - Realistic contact handling
 - Overall faster and more efficient in simulation of low-level physics
 - Less flexible than in handling integration of high-level sensors (camera, gps) and navigation
- <https://mujoco.org/>



- Contains a collection of robot descriptions for all Unitree robots
- Integrates directly the Unitree sdk 2 API into MuJoCo
 - The robot replicates exactly (almost) all possible API commands



https://github.com/unitreerobotics/unitree_mujoco

- A kinematic whole body planner can be seen as the Closed Loop Inverse Kinematics algorithm for floating base robots
- Given its structure, both the torso and the pelvis could work as floating base for H1-2, let's choose the torso from now on
- Let's assume that we want our robot to stand still, maintain its balance statically, lean slightly forwards, and aim for a specific position and orientation with its right arm. How do we translate these requirements to the task space of the robot?

- Let's assume that we want our robot to stand still and don't slip, maintain its balance statically, lean slightly forwards, and aim for a specific position and orientation with its right arm.
- **Stand still and don't slip:** The 6D components of both feet must be regulated
- **Static balance:** The CoM must stay within the support polygon
- **Lean forward:** The torso orientation must be regulated
- **Right arm:** The 6D components of the arm must be regulated

- **Stand still and don't slip:** The 6D components of both feet must be regulated
- **Static balance:** The CoM must stay within the support polygon
- **Lean forward:** The torso orientation must be regulated
- **Right arm:** The 6D components of the arm must be regulated

$$\dot{x} = \begin{bmatrix} -K_{p,com}e_{com} \\ -K_{p,lf}e_{lf} \\ -K_{p,rf}e_{rf} \\ -K_{p,torso}e_{torso} \\ -K_{p,rh}e_{rh} \end{bmatrix} \in \mathbb{R}^{24}$$

- We follow an approach similar to CLIK $\dot{q} = J^\dagger \dot{x}$ $n_v = 6 + n_j$

$$J = \begin{bmatrix} J_{com} \\ J_{lf} \\ J_{rf} \\ J_{torso} \\ J_{rh} \end{bmatrix}$$

$$J_{com} \in \mathbb{R}^{3 \times n_v}$$

$$J_{lf} \in \mathbb{R}^{6 \times n_v}$$

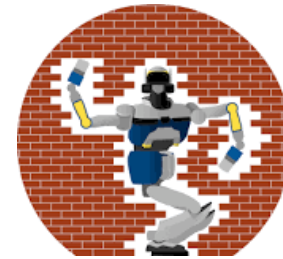
$$J_{rf} \in \mathbb{R}^{6 \times n_v}$$

$$J_{torso} \in \mathbb{R}^{3 \times n_v}$$

$$J_{rh} \in \mathbb{R}^{6 \times n_v}$$

$$J^\dagger = J^T (JJ^T + \lambda^2 I)$$

- How do we compute the previous jacobians and 6D errors in practice?
- **Pinocchio** is a library for efficiently computing the dynamics (and derivatives) of articulated rigid-body models



Pinocchio
Efficient and versatile rigid body dynamics algorithms

- We should also add secondary tasks in the null space of the primary tasks
 - Posture regularization $\dot{q}_s = K_{posture}(q_{nom} - q)$
 - Joint limit avoidance (!)

$$\dot{q} = J^\dagger \dot{x}_p + N \dot{q}_s \quad N = I - J^\dagger J$$

- In practice, both the kinematic planner and the low-level controller run in the same loop cycle
- The low-level controller is accessed via its interface that requires to set the following for each joint of the robot:
 - Target position
 - Target velocity
 - Proportional gain
 - Derivative gain
 - Feedforward torque

$$\tau = K_p(q_{ref} - q) + K_d(\dot{q}_{ref} - \dot{q}) + \tau_{ff}$$